

Slowly Changing Dim Notebook

New notebook

SCD Type 2 Production Template

Below is a **clean, production-ready, Databricks/Fabric** notebook template that performs **Slowly Changing Dimension Type 2 (SCD2)** handling using **Delta Lake MERGE**. This is the **industry-standard** implementation used in enterprise lakehouse architectures.

It supports:

- **Insert new dimension rows**
- **Expire old rows** when a change is detected
- **Insert new SCD2 version** with updated fields
- **Natural key-based matching**
- **Versioning and effective dates**

You can use this notebook as a **production template** immediately.

SCD Type 2 — Delta Lake Notebook Template

1. Parameters

If you want to get current Lakehouse info (friendly name)

```
In [25]: import os
from notebookutils import mssparkutils

print(os.getcwd())

row = spark.sql("SELECT current_catalog(), current_database()").collect()[0]
print(row)

catalog = row[0]
lakehouse = row[1]

current_lakehouse_id = row[0]
print("ID : ", current_lakehouse_id)

print("Catalog:", catalog)
print("Lakehouse Name:", lakehouse)

#print("Lakehouse ID:", lakehouse_id)
#print("Lakehouse Name:", Lh["name"])

# from config setting
#print(spark.conf.get("spark.sql.catalogImplementation"))
#print(spark.conf.get("spark.sql.defaultDatabase")) #not ca
```

StatementMeta(, 4276fe30-aa96-4f52-a3ba-1fdc7f8a9cd8, 27, Finished, Available, Finished, False)

```
/mnt/var/hadoop/tmp/nm-local-dir/usercache/trusted-service-user/appcache/applicat
ion_1770245392320_0001/container_1770245392320_0001_01_000001
Row(current_catalog()='spark_catalog', current_database()='laserengravelakehous
e')
ID : spark_catalog
Catalog: spark_catalog
Lakehouse Name: laserengravelakehouse
```

```
In [26]: row = spark.sql("SELECT current_database()").collect()[0]
print("Spark Lakehouse ID:", row[0])

from notebookutils import mssparkutils

lakehouse_id = spark.sql(
    "SELECT current_database()"
).collect()[0][0]

lh = mssparkutils.lakehouse.get(lakehouse_id)

#print("Lakehouse name:", lh["name"])
print("Lakehouse id:", lh["id"])
print("Workspace id:", lh["workspaceId"])
```

```
StatementMeta(, 4276fe30-aa96-4f52-a3ba-1fdc7f8a9cd8, 28, Finished, Available, Fi
nished, False)
Spark Lakehouse ID: laserengravelakehouse
Lakehouse id: 8af43c6b-4303-4b0b-8459-7621117771f2
Workspace id: 4ecbaf18-d748-4bd3-b6cd-5743f135f2bf
```

If you want to get current Lakehouse info (friendly name)

Source table (staging area / raw feed) and Target Location

```
In [27]: source_path = "Files/staging/customers"

# Target SCD2 dimension table
target_path = "Files/dim/customer_dim"
target_table = "dim_customer"
```

```
StatementMeta(, 4276fe30-aa96-4f52-a3ba-1fdc7f8a9cd8, 29, Finished, Available, Fi
nished, False)
```

Business Key (could be a compound key - multiple columns for row uniqueness)

```
In [28]: # Natural key for SCD2
business_key = "customer_id"
```

```
StatementMeta(, 4276fe30-aa96-4f52-a3ba-1fdc7f8a9cd8, 30, Finished, Available, Fi
nished, False)
```

Type 2 SCD Change Columns to track - So any change in any of these columns would flag a new 'insert' for the customer

and "expire" their previous row (populate effective_end date

```
In [29]: # Columns used to detect changes (non-key columns)
scd_columns = ["customer_name", "customer_status", "address"]
```

```
StatementMeta(, 4276fe30-aa96-4f52-a3ba-1fdc7f8a9cd8, 31, Finished, Available, Fi
nished, False)
```

```
In [30]: # Metadata columns for SCD2
effective_start = "effective_start_date"
effective_end = "effective_end_date"
is_current = "is_current"

now = "current_timestamp()"
```

StatementMeta(, 4276fe30-aa96-4f52-a3ba-1fdc7f8a9cd8, 32, Finished, Available, Finished, False)

2. Load Incoming Staging Data - load to memory in temporary table

```
In [31]: df_source = spark.read.format("delta").load(source_path).orderBy("customer_id")
df_source.createOrReplaceTempView("src")

print("Source count:", df_source.count())
```

StatementMeta(, 4276fe30-aa96-4f52-a3ba-1fdc7f8a9cd8, 33, Finished, Available, Finished, False)

Source count: 200

Let's see what we've staged in the temp table now

```
In [ ]: df_temp = spark.sql("SELECT * FROM src LIMIT 1000").orderBy("customer_id")
display(df_temp)
```

3. Debug - table locations

```
In [ ]: print("target table:", target_table)
print("target table path:", target_path)
```

Create target delta table if does not exist

```
In [ ]: spark.sql(f"""
CREATE TABLE IF NOT EXISTS {target_table} (
    customer_id STRING,
    customer_name STRING,
    customer_status STRING,
    address STRING,
    {effective_start} TIMESTAMP,
    {effective_end} TIMESTAMP,
    {is_current} BOOLEAN
)
USING DELTA
""")
```

4. Define MERGE Logic (SCD2 Core Logic)

This is the production pattern.

The "OR" join(...): "OR" is specified as the part that joins the iterated values returned from the list comprehension (expression resolved from the list)

Concatenates them into one single string, inserting " OR " between each element

"tgt.customer_name <> src.customer_name OR tgt.address <> src.address OR tgt.status <> src.status"

```
In [35]: spark.sql(f"""
MERGE INTO {target_table} AS tgt
USING (
    SELECT
        *,
        current_timestamp() AS update_ts
    FROM src
) AS src
ON tgt.{business_key} = src.{business_key} AND tgt.{is_current} = TRUE

WHEN MATCHED AND (
    {" OR ".join([f"tgt.{c} <> src.{c}" for c in scd_columns])}
)
THEN UPDATE SET
    tgt.{effective_end} = src.update_ts,
    tgt.{is_current} = FALSE

WHEN NOT MATCHED THEN INSERT (
    {business_key},
    {"", ".join(scd_columns)}},
    {effective_start},
    {effective_end},
    {is_current}
)
VALUES (
    src.{business_key},
    {"", ".join([f"src.{c}" for c in scd_columns])},
    src.update_ts,
    NULL,
    TRUE
)
""")
```

StatementMeta(, 4276fe30-aa96-4f52-a3ba-1fdc7f8a9cd8, 37, Finished, Available, Finished, False)

Out[35]: DataFrame[num_affected_rows: bigint, num_updated_rows: bigint, num_deleted_rows: bigint, num_inserted_rows: bigint]

```
In [ ]: spark.sql("""
SELECT *
FROM dim_customer
ORDER BY customer_id
""").show()
```

5. SCD2 Logic Explained Quickly

✓ When values change:

1. The current record is **closed**

* `effective_end_date = current_timestamp()`

- `is_current = false`

2. A new record is **inserted**

```
* effective_start_date = current_timestamp()
```

- `is_current = true`

✓ When values do NOT change:

- No modifications are made.

✓ When a business key does not exist:

- A new SCD2 record is inserted.

6. Check Final Results

```
In [ ]: df_dim = spark.read.format("delta").load(target_path)
display(df_dim.orderBy("customer_id", effective_start))
```

Check statuses

```
In [ ]: spark.sql("""
SELECT *
FROM dim_customer
WHERE is_current = TRUE
ORDER BY customer_id, effective_start_date
""").show()
```

WHERE is_current = FALSE

```
In [ ]: spark.sql("""
SELECT *
FROM dim_customer
WHERE is_current = FALSE
ORDER BY customer_id, effective_start_date
""").show()
```

```
In [ ]: # Count number of changes per customer

spark.sql("""
SELECT customer_id, COUNT(*) AS num_versions
FROM dim_customer
GROUP BY customer_id
HAVING COUNT(*) > 1
ORDER BY num_versions DESC
""").show()
```

Compare consecutive rows for each customer_id to see which attributes changed.

```
In [ ]: spark.sql("""
SELECT a.customer_id, a.customer_name, b.customer_name as customer_name_stg, a.c
FROM dim_customer a left outer join (select * from src) b on b.customer_id = a.c
WHERE a.is_current = FALSE
ORDER BY a.customer_id, a.effective_start_date
""").show()
```

```
# Compare consecutive rows for each customer_id to see which attributes changed.
```

```
In [ ]: spark.sql("select * from src").orderBy("customer_id").show()
```

```
In [ ]: spark.sql("""
SELECT customer_id, customer_name, customer_status, effective_start_date
FROM dim_customer
WHERE effective_end_date IS NOT NULL
ORDER BY effective_end_date DESC
""").show()
# Lists only the rows that were replaced by a new version.
```

Python / pandas approach (optional)

df_changes now contains all actual changes in the dimension.

```
In [ ]: import pandas as pd

# Assume df is already loaded from Spark to Pandas
df = spark.table("dim_customer").toPandas()

# Sort by customer and effective date
df.sort_values(["customer_id", "effective_start_date"], inplace=True)

# Identify previous values per customer
df['prev_name'] = df.groupby('customer_id')['customer_name'].shift(1)
df['prev_status'] = df.groupby('customer_id')['customer_status'].shift(1)

# Filter only rows where something changed
df_changes = df[
    (df['customer_name'] != df['prev_name']) |
    (df['customer_status'] != df['prev_status'])
]

# Optional: view the first 40 rows
df_changes.head(40)
```

7. Return Success to Pipeline

```
In [ ]: from notebookutils import mssparkutils
mssparkutils.notebook.exit("SCD2_SUCCESS")
```

This is a fully working SCD Type 2 Delta Lake notebook.

It follows production best practices:

- Proper MERGE logic
- Versioning metadata
- Automatic start/end timestamps
- Change detection only on relevant columns
- Handles inserts + updates + new versions safely

Clear out dimension table for test

```
In [ ]: spark.sql(""" truncate table dim_customer """)
```